

WAP a C to write the (i) FCFS (ii) SJF

→ (i) FCFS

```
#include <stdio.h>
```

```
struct process {
```

```
    int pid;
```

```
    int at;
```

```
    int bt;
```

```
    int ct;
```

```
    int tati;
```

```
    int wt;
```

```
    int rt;
```

```
};
```

```
int main () {
```

```
    int n, i, j;
```

```
    struct process p[20], temp;
```

```
    float total ct = 0, total tat = 0, total wt = 0, total rt = 0;
```

```
    printf("Enter number of processes:");
```

```
    scanf("%d", &n);
```

```
    for (i = 0; i < n; i++) {
```

```
        p[i].pid = i + 1;
```

```
        printf("Enter Arrival Time and Burst Time for
```

```
        (n+1) Process %d: ", p[i].pid);
```

```
        scanf("%d %d", &p[i].at, &p[i].bt);
```

```
    }
```

```
    for (i = 0; i < n - 1; i++) {
```

```
        for (j = i + 1; j < n; j++) {
```

```
            temp = p[i];
```

```
            p[i] = p[j];
```

```
            p[j] = temp;
```



```

printf("\n PIPLEAT LTA L OF L T A C L T (L T L K T K)");
for(i=0;i<n;i++)
    printf("%d\t", d[i]);
    P(i) P(d, R, s P(i), B, s P(i), c, s R(i) T A,
    R(i), L T, s P(i) L T);
}

```

3

OIP

Enter no. of process: 4

Enter Arrival Time and Burst Time for process 1:

0 7

Enter Arrival Time & burst Time for process 2

8 3

Enter Arrival Time & burst Time for process 3:

3 2

Enter Arrival Time & burst Time for process 4:

5 6

PTA	AT	BT	CT	TAT	WT	RT
P ₁	0	7	7	7	6	0
P ₂	3	2	9	6	4	7
P ₃	5	2	15	10	4	4
P ₄	8	3	18	16	7	7

P₁ 0 7 7 7 6 0P₂ 3 2 9 6 4 7P₃ 5 2 15 10 4 4P₄ 8 3 18 16 7 7

Average Turnaround Time = 9.25

Average Waiting Time = 3.75

Average Response Time = 3.75

27/12/20

SR JFC

#include <stdio.h>

struct process {

int pid, at, btch, turn, wt, rt;

; int remaining, completed, sorted;

};

int main () {

int n, choice;

struct process p[20];

float total_wt = 0, total_rt = 0, total_bt = 0;

printf("Enter number of process: ");

scanf("%d", &n);

for (int i = 0; i < n; i++) {

p[i].pid = i + 1;

printf("Enter AT & BT of p. %d", p[i].pid);

scanf("%d %d", &p[i].at & p[i].bt);

p[i].remaining = p[i].bt;

p[i].completed = 0;

p[i].started = 0;

printf("Ln 1. SJF Non-Premptive");

printf("Ln 2. SJF Premptive SKJF");

printf("Enter choice");

scanf("%d", &choice);

int current_time = 0, completed = 0;

if (choice == 1) {

while (completed < n) {

int idn = 1;

int minPT = 9999;


```
for (int i = 0; i < n; i++)
```

```
    IF (R(i).at <= CurrentTime & R(i).Completed == 0)
```

```
        IF (P(i).at <= Limit)
```

```
            minDt = R(i).bt;
```

```
            idx = i;
```

```
        }
```

```
    }
```

```
}
```

```
IF (idx == -1)
```

```
{
```

```
    P(idx).at = CurrentTime - B(idx).at; //
```

```
    CurrentTime = P(idx).bt;
```

```
    P(idx).at = CurrentTime;
```

```
    P(idx).t.at = P(idx).cf * P(idx).at;
```

```
    P(idx).lt = P(idx).t.at - P(idx).bt;
```

```
    total.ct = P(idx).t;
```

```
    total.tot = P(idx).tot;
```

```
    total.Lat = P(idx).Lat;
```

```
    total.co = P(idx).Dt;
```

```
    P(idx).Completed = 1;
```

```
    Completed;
```

```
else
```

```
{
```

```
    CurrentPeriod;
```

```
}
```

```
}
```

```
}
```

```
else if (choice == 1)
```

```
    while (Completed < n)
```

```
        exit idx == 1;
```



```
int minK = 0;
```

```
for (int i = 0; i < n; i++) {
```

```
    IF (R[i].ct == currentTime) {
```

```
        remaining = 0;
```

```
    IF (P[i].remaining < minK) {
```

```
        minK = P[i].remaining;
```

```
        idx = i;
```

```
    }
```

```
}
```

```
}
```

```
IF (idx != -1) {
```

```
    IF (P[idx].start == 0) {
```

```
        P[idx].ct = completeTime - P[idx].ct;
```

```
        total.ct = P[idx].ct + (idx + 1);
```

```
        P[idx].started = 1;
```

```
    }
```

```
    P[idx].remaining = 1;
```

```
    current time = 1;
```

```
    IF (P[idx].remaining == 0)
```

```
        P[idx].ct = current time;
```

```
        P[idx].ct = P[idx].ct + P[idx].en
```

```
        P[idx].ct = P[idx].ct + P[idx].ht;
```

```
        total.ct = P[idx].ct;
```

```
        total.ct = P[idx].ct;
```

```
        total.ct = P[idx].ct;
```

```
    completed;
```

```
}
```

```
}
```

```
else {
```

```
    current time = 1;
```

```
}
```


return 0;

3

for $(n_1, n_2, n_3) \in \mathbb{N}^3$

$$p(1) \cdot a_1(1) \cdot b_1(1) \cdot c_1 \cdot p(1) \cdot p(1) \cdot \text{end} \cdot 11(2,2)$$

3

plant¹² Average TAF = 1.25¹² (all 1-0.81n)

Quintic: In Average RT = 1.2 f", total (at) n:

return 0;

3

Order no. 69 Process: 1. ...

Enter AT and P70, P1: 0 7

Enter AS and BS too 11:8 3

Enter AT and of for p3! 3 2

Enter of cash BT for by. S.

2. 31st Nov - promissive

2.53f Presumptive (SEJF)

Enter choice: 1

Read AT BT CT TAC WT AT

0 6 7 7 7 6 6

p2 0 3 12 7 1 1

12 3 2 9 6 7 4

$\begin{matrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 \\ 1 & 5 & 6 & 18 & 13 & 2 & 2 & 1 \end{matrix}$

Average ~~Time~~ = 7.50

Average WT = 3.03

Average RT = 3.03

Enter choice : 12

	PSD	AT	BT	CT	AT	WT	RT
P1	0	7	9	9	2	0	
P2	0	3	12	7	1	1	
P3	3	2	5	2	0	0	
P4	5	6	18	13	7	3	

Average TAT = 7.00

Average WT = 2.50

Average RT = 2.03

Here Average waiting time of and
 PAF of SJF preemptive is lower than
 non preemptive so it is better.

27/3/20

* Priority

```
#include <stdio.h>
```

```
struct process {
```

```
    int pid, at, bt, spz;
```

```
    int rt, stat, wt;
```

```
    int remaining;
```

```
    int completed;
```

```
};
```

```
int main() {
```

```
    int n, choice;
```

```
    struct process p[20];
```

```
    float total_tat = 0, total_wt = 0;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    for (int i = 0; i < n; i++) {
```

```
        p[i].pid = i + 1;
```

```
        printf("Enter AT, BT and priority for P%d:",
```

```
               "P[i].pid");
```

```
        scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].pr);
```

```
        p[i].remaining = p[i].bt;
```

```
        p[i].completed = 0;
```

```
    }
```

```
    printf("1. Priority Non-Preemptive");
```

```
    printf("2. Priority Preemptive");
```

```
    printf("Enter choice: ");
```

```
    scanf("%d", &choice);
```

```
    int current_time = 0, completed = 0;
```

```
    if (choice == 1) {
```

```
        while (completed < n) {
```

```
            int idx = -1;
```

```
            int highpr = 9999;
```



```

for (int i = 0; i < n; i++) {
    if (p[i].ct <= currentTime && p[i].comple-
        -ted == 0) {
        if (p[i].pt < high pt) {
            high pt = p[i].pt;
            idx = i;
        }
    }
}

```

```

if (idx != -1) {
    currentTime += p[idx].bt;
    p[idx].ct = currentTime;
    p[idx].tot = p[idx].ct - p[idx].ct;
    p[idx].wt = p[idx].tot - p[idx].bt;
    total_tct += p[idx].tot;
    total_wt += p[idx].wt;
    p[idx].completed = 1;
}
else {
    // ...
}
}

```

```

// ...
else if (choice == 2) {
    while (completed < n) {
        int idx = ...;
        int high pt = ...;
        for (int i = 0; i < n; i++) {
            if (p[i].ct <= currentTime && p[i].
                remaining > 0) {
                if (p[i].pt < high pt) {
                    high pt = p[i].pt;

```



```
idx = i;
```

```
}
```

```
}
```

```
}
```

```
if (idx != -1) {
```

```
    p[idx].remaining =
```

```
    (current time - 1);
```

```
if (p[idx].remaining == 0) {
```

```
    p[idx].ct = current time;
```

```
    p[idx].tct = p[idx].ct - p[idx].bt;
```

```
    p[idx].wt = p[idx].tct - p[idx].bt;
```

```
    total_tct += p[idx].tct;
```

```
    total_wt += p[idx].wt;
```

```
    completed++;
```

```
}
```

```
else {
```

```
    current time++;
```

```
}
```

```
}
```

```
else {
```

```
    printf("Invalid choice");
```

```
    return 0;
```

```
}
```

```
printf("Input task BT, PR, CT\n");
```

```
for (int i = 0; i < n; i++) {
```

```
    printf("P: %d, BT: %d, PR: %d, CT: %d\n",
```

```
        p[i], bt[i], pr[i], ct[i]);
```

```
for (int i = 0; i < n; i++) {
```

```
    p[i].bt = bt[i], p[i].pr = pr[i], p[i].ct = ct[i];
```


$\{ p(i), (t \cdot p(i)), (t \cdot s \cdot p(i)), w(i) \}$

print("Average Turnaround Time
= 1.24", total, t/n);

print("Average Waiting Time = 1.24
in", total, w/n);

return 0;

3

Output:

Processes: 5

AT	BT	PR	For	P1:0	1	0	3	5	3	0	3	0
3					2	2	2	3	11	7	9	7
5					3	3	5	2	8	0	5	0

AT BT PR For P2:2

2					4	4	4	4	15	7	11	7
3					5	6	1	1	9	2	3	2

AT BT PR For P3:3

5												
2												

AT BT PR For P4:4

4												
4												

AT BT PR For P5:6

1												
1												

1. Priority Non-preemptive

2. Priority Preemptive

Choice 1

Worst case:

P1	P2	P3	P4	P5	P2	P4	P1
0	3	8	9	11	15		

* LAB-3 (Round robin)

```
#include <stdio.h>
```

```
struct process {
```

```
    int pid, at, bt, ct, stat, start, stop;
```

```
};
```

```
int main() {
```

```
    int n, stop;
```

```
    struct process p[20];
```

```
    int queue[100];
```

```
    int visited[20] = {0};
```

```
    int started[20] = {0};
```

```
    int front = 0, rear = 0;
```

```
    int time = 0, completed = 0;
```

```
    float avgwt = 0, avgct = 0, avgbt = 0;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    for (int i = 0; i < n; i++) {
```

```
        p[i].pid = i+1;
```

```
        printf("Process %d\n", i+1);
```

```
        printf("Arrival Time: ");
```

```
        scanf("%d", &p[i].at);
```

```
        printf("Burst Time: ");
```

```
        scanf("%d", &p[i].bt);
```

```
        p[i].ct = p[i].bt;
```

```
        p[i].res = 1;
```

```
    }
```

```
    for (int i = 0; i < n-1; i++) {
```

```
        for (int j = i+1; j < n; j++) {
```

```
            if (p[i].at > p[j].at) {
```

```
                struct process temp = p[i];
```

```
                p[i] = p[j];
```



```
p(i) = temp;
```

```
}
```

```
}
```

```
printf("Enter Time Quantum: ");
```

```
scanf("%d", &tq);
```

```
while (completed < n) {
```

```
    for (int i = 0; i < n; i++) {
```

```
        if (p(i).ct <= tq && visited(i) == 0) {
```

```
            queue[rear++] = i;
```

```
            visited(i) = 1;
```

```
        }
```

```
    }
```

```
    if (front == rear) {
```

```
        time += tq;
```

```
        continue;
```

```
    int idx = queue[front++];
```

```
    if (started[idx] == 0) {
```

```
        p[idx].res = time - p[idx].id;
```

```
        a[idx++] = p[idx].res;
```

```
        started[idx] = 1;
```

```
    }
```

```
    if (p[idx].ct > tq) {
```

```
        p[idx].ct -= tq;
```

```
        time += tq;
```

```
    }
```

```
    else {
```

```
        time += p[idx].ct;
```

```
        p[idx].ct = 0;
```

```
        p[idx].ct = time;
```

```
        p[idx].tot = p[idx].ct + p[idx].at;
```



```

p[idx].wt = p[idx].tot - p[idx].bt;
avg_wt += p[idx].wt;
avg_tot += p[idx].tot;
Completed++;
}

```

```

for (int i = 0; i < n; i++) {
    if (p[i].at < 2 * time & visited[i] == 0) {
        queue (rear++) = i;
        visited[i] = 1;
    }
}

```

```

if (p[idx].at > 0) queue (rear++) = idx;
}

```

```

printf("Round Robin scheduling - 10");
printf("Process P1 P2 P3 P4 P5 P6 P7 P8 P9 P10\n");

```

```

for (int i = 0; i < n; i++) {
    printf("P: %.1d %.1d %.1d %.1d %.1d %.1d %.1d %.1d %.1d %.1d\n",
           p[i].pid, p[i].at, p[i].bt, p[i].et, p[i].tot,
           p[i].wt, p[i].res);
}

```

```

printf("Average RT = %.1f, Avg tot = %.1f\n",
       avg_tot / n, avg_wt / n);
printf("Average WT = %.1f, Avg res = %.1f\n",
       avg_wt / n, avg_res / n);
return 0;
}

```

```

printf("Average RT = %.1f, Avg tot = %.1f\n",
       avg_tot / n, avg_wt / n);
printf("Average WT = %.1f, Avg res = %.1f\n",
       avg_wt / n, avg_res / n);
return 0;
}

```

~~main~~

Output:-

Enter number of Process: 3

Process 1

Arrival Time: 0

Burst Time: ~~5~~ 5

Process 2

Arrival Time: 3

Burst Time: 2

Process 3

Arrival Time: 9

Burst Time: 7

Enter Time Quantum: 5

--- Round Robin Scheduling ---

Process	AT	BT	CT	TAT	WT	RT
P1	0	5	5	5	0	0
P2	3	2	24	17	9	2
P3	9	7	22	13	6	1

Average TAT = 13.00

Average WT = 5.00

Average RT = 1.00

Enter Time Quantum: 15

--- Round Robin Scheduling ---

Process	AT	BT	CT	TAT	WT	RT
P1	0	5	5	5	0	0
P2	3	2	17	14	2	2
P3	9	7	27	15	8	8

Average TAT = 11.33

Average WT = 3.33

Average RT = 3.33

If we increase the time quantum then average TAT, WCT, RT decrease and system become more efficient.

(EXAMPLE)

(P1) (P2) (P3) (P4) (P5) (P6) (P7) (P8) (P9) (P10)

10 = 10000 10 = 10000 10 = 10000 10 = 10000 10 = 10000

10 = 10000 10 = 10000 10 = 10000 10 = 10000 10 = 10000

10 = 10000 10 = 10000 10 = 10000 10 = 10000 10 = 10000

10 = 10000 10 = 10000 10 = 10000 10 = 10000 10 = 10000

10 = 10000 10 = 10000 10 = 10000 10 = 10000 10 = 10000

10 = 10000 10 = 10000 10 = 10000 10 = 10000 10 = 10000

10 = 10000 10 = 10000 10 = 10000 10 = 10000 10 = 10000

10 = 10000 10 = 10000 10 = 10000 10 = 10000 10 = 10000

10 = 10000 10 = 10000 10 = 10000 10 = 10000 10 = 10000

10 = 10000 10 = 10000 10 = 10000 10 = 10000 10 = 10000

10 = 10000 10 = 10000 10 = 10000 10 = 10000 10 = 10000

10 = 10000 10 = 10000 10 = 10000 10 = 10000 10 = 10000

10 = 10000 10 = 10000 10 = 10000 10 = 10000 10 = 10000

10 = 10000 10 = 10000 10 = 10000 10 = 10000 10 = 10000

10 = 10000 10 = 10000 10 = 10000 10 = 10000 10 = 10000

10 = 10000 10 = 10000 10 = 10000 10 = 10000 10 = 10000

10 = 10000 10 = 10000 10 = 10000 10 = 10000 10 = 10000

10 = 10000 10 = 10000 10 = 10000 10 = 10000 10 = 10000

10 = 10000 10 = 10000 10 = 10000 10 = 10000 10 = 10000

10 = 10000 10 = 10000 10 = 10000 10 = 10000 10 = 10000

10 = 10000 10 = 10000 10 = 10000 10 = 10000 10 = 10000

10 = 10000 10 = 10000 10 = 10000 10 = 10000 10 = 10000

10 = 10000 10 = 10000 10 = 10000 10 = 10000 10 = 10000

N/A
22/3/20

* Multilevel :-

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100
typedef struct {
    int id, at, bt, rt, type, (1's wt & tat);
} process;
process P[MAX];
int sysQ[MAX], userQ[MAX];
int frontS = 0, rearS = 0;
int frontU = 0, rearU = 0;
void enqueueSys(int x) {
    sysQ[rearS++] = x;
}
int dequeueSys() {
    return sysQ[frontS++];
}
void enqueueUser(int x) {
    userQ[rearU++] = x;
}
int dequeueUser() {
    return userQ[rearU++];
}
int isSysEmpty() {
    return frontS == rearU;
}
int isUserEmpty() {
    return frontU == rearU;
}
void Sort(process P[], int n) {
    process temp;
    for (int i = 0; i < n - 1; i++) {
```



```
if (p[j].at > p[j+1].at) {
```

```
    temp = p[j];
```

```
    p[j] = p[j+1];
```

```
    p[j+1] = temp;
```

```
}
```

```
}
```

```
}
```

```
int main() {
```

```
    int n;
```

```
    printf("Enter number of processes:");
```

```
    scanf("%d", &n);
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf("Enter process %d id: ", i);
```

```
        p[i].id = i;
```

```
        printf("Enter arrival time: ");
```

```
        scanf("%d", &p[i].at);
```

```
        printf("Enter burst time: ");
```

```
        scanf("%d", &p[i].bt);
```

```
        printf("Enter type (0 = System, 1 = User): ");
```

```
        scanf("%d", &p[i].type);
```

```
        p[i].rt = p[i].bt;
```

```
}
```

```
    Sort(p, n);
```

```
    int time = 0, completed = 0, i = 0;
```

```
    int current = -1;
```

```
    while (completed < n) {
```

```
        while (run && p[i].at <= time) {
```

```
            if (p[i].type == 0)
```

```
                enqueueSys(i);
```

```
            else
```

```
                enqueueUser(i);
```



```

printf("In 3D type 1 for 1st, 2 for 2nd, 3 for 3rd\n");
for(int i=0; i<n; i++)
{
    printf("1. att: %d, 2. att: %d, 3. att: %d\n",
           p[i].att, p[i].att, p[i].att);
    p[i].id = i;
    p[i].type = 0; // "System" // "User"
    p[i].att = 1;
    p[i].bt = 1;
    p[i].ct = 1;
    p[i].wt = 1;
    p[i].tt = 1;
    avgWT += p[i].wt;
    avgTAT += p[i].tt;
}
printf("Average Waiting time = %.2f\n",
       avgWT/n);
printf("Average Turnaround Time = %.2f\n",
       avgTAT/n);
return 0;
}

```

Output :-

Enter number of processes: 4

Process 0

Enter arrival time: 0

Enter burst time: 5

Enter type (0 = System, 1 = User): 0

Process 1

Enter arrival time: 2

Enter burst time: 3

Enter type (0 = System, 1 = User): 1

Process 2

Enter arrival time: 1

Enter burst time: 4


```

printf("In 3d Type TAT = TBT + CT + WBT + ET\n");
for(int i = 0; i < n; i++) {
    printf("%d. dt: %d. st: %d. dt: %d. at: %d. dt: %d.\n",
        p[i].id,
        (p[i].type == 0) ? "System" : "User",
        p[i].at,
        p[i].bt,
        p[i].ct,
        p[i].wt,
        p[i].tat);
    avgWT += p[i].wt;
    avgTAT += p[i].tat;
}
printf("Average Waiting Time = %.2f",
    avgWT / n);
printf("In Average Turnaround Time = %.2f",
    avgTAT / n);
return 0;
}

```

Output :-

Enter number of processes: 4

Process 0

Enter arrival time: 0

Enter burst time: 5

Enter type (0 = System, 1 = User): 0

Process 1

Enter arrival time: 2

Enter burst time: 3

Enter type (0 = System, 1 = User): 1

Process 2

Enter arrival time: 1

Enter burst time: 4

Enter type (0 = System, 1 = User): 0
Process?

Enter arrival time: 5

Enter burst time: 2

Enter type (0 = System, 1 = User): 1

ID	Type	AT	BT	CT	WT	TAT
0	System	0	5	5	0	5
2	System	1	4	9	4	9
1	User	2	3	12	2	10
3	User	3	2	14	9	11

Average Waiting Time = 5.00

Average Turnaround Time = 8.50

Description

27/10/22

* Write a C program to simulate real-time CPU Scheduling algorithms in C.

a.) Rate-Monotonic

b.) Earliest-deadline First

c.) Proportional scheduling

→ #include <stdio.h>

#include <math.h>

struct Process {

int id; // id of process

int burst; // burst time

int deadline; // deadline

int period; // period

int completion; // completion time

int waiting; // waiting time

int turnaround; // turnaround time

};

void swap (struct Process *a, struct Process *b)

{

struct Process temp = *a;

*a = *b;

*b = temp;

}

void sortByDeadline (struct Process p[], int n)

{

for (int i = 0; i < n - 1; i++)

{

for (int j = i + 1; j < n; j++)

{

if (p[i].deadline > p[j].deadline)

{

swap (&p[i], &p[j]);

}

}

}

}


```

void sortByPeriod (struct process p[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            if (p[j].period > p[j+1].period)
                swap(p[j], p[j+1]);
        }
    }
}

```

```

3
3
3

```

```

void effScheduling (struct process p[], int n)
{
    struct process temp[100];
    for (int i = 0; i < n; i++)
        temp[i] = p[i];
    sortByDeadline (temp, n);
    int time = 0;
    for (int i = 0; i < n; i++)
    {
        time += temp[i].burst;
        temp[i].completion = time;
        temp[i].turnaround = temp[i].completion;
        temp[i].waiting = temp[i].turnaround - temp[i].burst;
    }
}

```

float utilization = 0;

```

for (int i = 0; i < n; i++)
    utilization += (float) temp[i].burst / temp[i].deadline;
printf("In ==> Earliest Deadline First (EDF) Scheduling ==> %f\n", utilization);
printf("CPU Utilization : %.2f\n", utilization);
if (utilization <= 1)

```

```

printf("Schedulable (Utilization <= 1)\n");
else

```

```

printf("Not schedulable\n");
printf("ID (BF) Deadline (T) Burst (B)\n");
for (int i = 0; i < n; i++)
{
    printf("%d\t%d\t%d\t%d\n", temp[i].id, temp[i].burst, temp[i].deadline, temp[i].completion);
}

```

```

3
3
3

```

```

void inScheduling (struct process p[], int n)
{
    struct process temp[100];
    for (int i = 0; i < n; i++)
        temp[i] = p[i];
    sortByPeriod (temp, n);
    int time = 0;
    for (int i = 0; i < n; i++)
    {
        time += temp[i].burst;
        temp[i].completion = time;
        temp[i].turnaround = temp[i].completion;
        temp[i].waiting = temp[i].turnaround - temp[i].burst;
    }
}

```

```

float utilization = 0;
for (int i = 0; i < n; i++)
    utilization += (float) temp[i].burst / temp[i].period;

```



```
printf("Schedulable Utilization = %f\n");
else
```

```
printf("Not Schedulable\n");
```

```
printf("ID | BF | Deadline | TC | WT | TAT | \n");
```

```
for (int i = 0; i < n; i++) {
```

```
printf("%d | %d | %d | %d | %d | %d | %d | \n",
```

```
temp[i].id,
```

```
temp[i].burst,
```

```
temp[i].deadline,
```

```
temp[i].completion,
```

```
temp[i].waiting,
```

```
temp[i].turnaround,
```

```
}
```

```
}
```

```
void minScheduling (struct Process p[], int n)
```

```
{
    struct Process temp[200];
```

```
for (int i = 0; i < n; i++)
```

```
temp[i] = p[i];
```

```
sort By Period (temp, n);
```

```
int time = 0;
```

```
for (int i = 0; i < n; i++) {
```

```
time += temp[i].burst;
```

```
temp[i].completion = time;
```

```
temp[i].turnaround = temp[i].completion
```

```
temp[i].waiting = temp[i].turnaround
```

```
temp[i].burst;
```

```
}
```

```
float utilization = 0;
```

```
for (int i = 0; i < n; i++)
```

```
utilization += (float) temp[i].burst /
```

```
temp[i].period;
```

```
printf("Utilization = %f\n", utilization);
```



```
float bound = 1.0 / (period - 1);
```

```
printf("Rate Monotonic
```

```
Scheduling (RM) is used\n");
```

```
printf("CPU Utilization: %.2f\n", utiliza-  
tion);
```

```
printf("RM Bound: %.4f\n", bound);
```

```
if (utilization < bound)
```

```
printf("Schedulable Utilization < RM Bound\n");
```

```
else
```

```
printf("Not Schedulable\n");
```

```
printf("SDBF: %d\n", SDBF);
```

```
for (int i = 0; i < n; i++)
```

```
printf("id: %d, d: %d, t: %d, p: %d, c: %d, w: %d, a: %d\n",
```

```
id, d, t, p, c, w, a);
```

```
temp[id] = burst;
```

```
temp[id] = period;
```

```
temp[id] = completion;
```

```
temp[id] = waiting;
```

```
temp[id] = turnaround;
```

```
3. Round Robin
```

```
3. Round Robin
```

```
3. Round Robin
```

```
3. Round Robin
```

```
printf("Enter number of process: ");
```

```
scanf("%d", &n);
```

```
struct process p[100];
```

```
printf("Enter process details: \n");
```

```
for (int i = 0; i < n; i++)
```

```
scanf("%d %d %d %d %d %d %d",
```

```
&p[i].id, &p[i].d, &p[i].t, &p[i].p, &p[i].c, &p[i].w, &p[i].a);
```



```

print("Burst Time:");
scanf("%d", &p[i].burst);
print("Deadline (for EDF):");
scanf("%d", &p[i].deadline);
print("Period (for RMS):");
scanf("%d", &p[i].period);
}

```

```

edf Scheduling(pn);
rm Scheduling(pn);
return 0;
}

```

Output:-

Enter number of processes: 3

Enter process details: 5 0

Process: 0

Burst Time: 3

Deadline (for EDF): 7

Period (for RMS): 20

Process: 1

Burst Time: 2

Deadline (for EDF): 4

Period (for RMS): 5

Process: 2

Burst Time: 2

Deadline (for EDF): 8

Period (for RMS): 10

=== Earliest Deadline First (EDF) Scheduling:

CPU utilization: 1.18

Not schedulable

ID BF Deadline CT WT TAT

1 2 10 14 2 0 7 10

0 3 7 5 2 5

2 2 10 2 7 5 7

== Rate Monotonic Scheduling (RMS) ==

CPU Utilization: $p + S$

RM Bound: 0.7798

Schedulable Utilization $\leq$ (RM Bound)

ID BF Period CT WT TAT

1 2 5 2 0 7 10

2 2 10 4 2 4 2

0 3 20 7 14 7 3

* Producer-Consumer problem using semaphore

→ #include <stdio.h>

#include <pthread.h>

#include <semaphore.h>

#include <unistd.h>

#define Buffer_SIZE 5

int buffer[Buffer_SIZE];

int in = 0, out = 0;

sem_t empty, full;

pthread_mutex_t mutex;

void * producer(void * arg)

{

int item;

for (item = 0; item < 5; item++)

{

sem_wait(&empty);

pthread_mutex_lock(&mutex);

buffer[in] = item;

printf("Produced: %d at buffer %d\n",

in = (in + 1) % Buffer_SIZE;

pthread_mutex_unlock(&mutex);

sem_post(&full);

sleep(2);

}

return NULL;

}

void * consumer(void * arg)

{

int item;

for (int i = 0; i < 5; i++)

{

* Producer-Consumer problem using semaphores.

→ #include <stdio.h>

#include <pthread.h>

#include <semaphore.h>

#include <unistd.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];

int in = 0, out = 0;

sem_t empty, full;

pthread_mutex_t mutex;

void * producer(void * arg)

{

int item;

for (item = 0; item < 15; item++)

{

sem_wait(&empty);

pthread_mutex_lock(&mutex);

buffer[in] = item;

printf("Produced: %d at buffer (%d)\n", item, in);

in = (in + 1) % BUFFER_SIZE;

pthread_mutex_unlock(&mutex);

sem_post(&full);

sleep(2);

}

return NULL;

}

void * consumer(void * arg)

{

int item;

for (int i = 0; i < 10; i++)

{


```

km_wait (Sfull);
pthread_mutex_lock (&mutex);
item = buffer [out];
printf ("Consumed: %d from buffer.\n",
        item, out);
out = (out + 1) % BUFFER_SIZE;
pthread_mutex_unlock (&mutex);
if (empty == 1)
    sleep (2);
}

```

```

return 0;
}
int main ()
{

```

```

    pthread_t p1, p2;
    km_wait (Sempty, 0, BUFFER_SIZE);
    km_wait (Sfull, 0, 0);
    pthread_mutex_init (&mutex, NULL);
    pthread_create (&p1, NULL, producer, NULL);
    pthread_create (&p2, NULL, consumer, 2);
    pthread_join (p1, NULL);
    pthread_join (p2, NULL);
    km_destroy (&empty);
    km_destroy (&full);
    pthread_mutex_destroy (&mutex);
    return 0;
}

```

Output:-

```

Produced: 0 at buffer [0]
Consumed: 0 from buffer [0]
Produced: 1 at buffer [1]
Consumed: 1 from buffer [1]
Produced: 2 at buffer [2]
Consumed: 2 from buffer [2]
Produced: 3 at buffer [3]
Consumed: 3 from buffer [3]
Produced: 4 at buffer [4]
Consumed: 4 from buffer [4]
Produced: 5 at buffer [0]
Consumed: 5 from buffer [0]
Produced: 6 at buffer [1]
Consumed: 6 from buffer [1]
Produced: 7 at buffer [2]
Consumed: 7 from buffer [2]
Produced: 8 at buffer [3]
Consumed: 8 from buffer [3]
Produced: 9 at buffer [4]
Consumed: 9 from buffer [4]
Produced: 10 at buffer [0]
Consumed: 10 from buffer [0]
Produced: 11 at buffer [1]
Consumed: 11 from buffer [1]
Produced: 12 at buffer [2]
Consumed: 12 from buffer [2]
Produced: 13 at buffer [3]
Consumed: 13 from buffer [3]
Produced: 14 at buffer [4]
Consumed: 14 from buffer [4]

```


Output:-

Produced: 0 at buffer [0]
 Consumed: 0 from buffer [0]
 Produced: 1 at buffer [1]
 Consumed: 1 from buffer [1]
 Produced: 2 at buffer [2]
 Produced: 3 at buffer [3]
 Consumed: 2 from Buffer [2]
 Produced: 4 at buffer [4]
 Produced: 5 at buffer [0]
 Consumed: 3 from buffer [3]
 Produced: 6 at buffer [2]
 Produced: 7 at buffer [2]
 Consumed: 4 from buffer [4]
 Produced: 8 at buffer [3]
 Produced: 9 at buffer [4]
 Consumed: 5 from buffer [0]
 Produced: 10 at buffer [0]
 Consumed: 6 from buffer [3]
 Produced: 11 at buffer [1]
 Consumed: 7 from buffer [2]
 Produced: 12 at buffer [2]
 Consumed: 8 from buffer [3]
 Produced: 13 at buffer [3]
 Consumed: 9 from buffer [4]
 Produced: 14 at buffer [4].

Trail :-

Time (S)	Active Thread	Producers	Consumers
0	Start	5	0
0	Producer	9	1
0	Consumer	5	0
1	Producer	7	1
2	Producer	3	2
2	Consumer	7	1
3	Producer	3	2
4	Producer	2	3
4	Consumer	3	2
5	Producer	2	3
6	Producer	1	4
6	Consumer	2	3
7	Producer	3	4
8	Producer	4	4
8	Consumer	4	0
9	Producer	0	0
10	Producer	0	0
10	Consumer	0	1

M.D.
 8/6/26

* Philosopher's Problem:

```
→ #include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <semaphore.h>
```

```
#include <unistd.h>
```

```
#define N 5
```

```
sem_t forks(N);
```

```
void *philosopher(void *num)
```

```
{ int id = *(int *) num;
```

```
while (1) {
```

```
    printf("Philosopher %d is thinking.\n", id);
```

```
    sleep(1);
```

```
    sleep(1);
```

```
    printf("Philosopher %d picked up left fork\n", id);
```

```
    sem_wait(&forks[id]);
```

```
    printf("Philosopher %d picked up right\n", id);
```

```
    fork[id+1];
```

```
    sem_wait(&forks[(id+1)%N]);
```

```
    printf("Philosopher %d is eating.\n", id);
```

```
    sleep(2);
```

```
    printf("Philosopher %d put down forks.\n", id);
```

```
    sem_post(&forks[id]);
```

```
    sem_post(&forks[(id+1)%N]);
```

```
    sleep(1);
```

```
    sleep(1);
```

```
}
```

```
}
```

```
int main()
```

```
{ pthread_t *ph(N);
```



```

int ids[N];
for (int i = 0; i < N; i++) {
    sem_init(&forks[i], 0, 1);
}
for (int i = 0; i < N; i++) {
    ids[i] = i;
    pthread_create(&ph[i], NULL, philosopher,
                  &ids[i]);
}
for (int i = 0; i < N; i++) {
    pthread_join(ph[i], NULL);
}
return 0;
}

```

Trace

Time Step	Philosopher	Action	States of Forks (F ₀ , F ₁ , F ₂ , F ₃)	State of
1	P ₀	Thinking	(1, 1, 1, 1)	
2	P ₁	Waits for fork	(1, 1, 1, 1)	
3	P ₁	Picks up fork 2 (right)	(1, 1, 0, 1)	
4	P ₁	Picks up fork 1 (left)	(1, 0, 0, 1)	
5	P ₁	Release, muller starts	(1, 0, 0, 0)	
6	P ₀	cross muller	(1, 0, 0, 0)	
7	P ₀	Picks up fork 1 (left)	(0, 0, 0, 0)	
8	P ₀	Blocked (waiting for fork 2)	(0, 0, 0, 0)	
9	P ₃	Waiting for muller	(0, 0, 0, 0)	

Available for Forks (0)
 taken (0) taken (0)
 taken (0) taken (0)
 taken (0)
 Available (2)
 taken (0)


```

* Banker's Algorithm
#include <stdio.h>
int main() {
    int n, m, i, j, k;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resource types: ");
    scanf("%d", &m);
    int alloc[n][m], max[n][m], need[n][m];
    int avail[m];
    printf("\nEnter Allocation Matrix: \n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < m; j++) {
            scanf("%d", &alloc[i][j]);
        }
    }
    printf("\nEnter Maximum Matrix: \n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < m; j++) {
            scanf("%d", &max[i][j]);
        }
    }
    printf("\nEnter Available Resources: \n");
    for (i = 0; i < m; i++) {
        scanf("%d", &avail[i]);
    }
    for (i = 0; i < n; i++) {
        for (j = 0; j < m; j++) {
            need[i][j] = max[i][j] - alloc[i][j];
        }
    }
}

```



```

int finish(n), safeSeq(n);
int work(m);
for(i=0; i<m; i++) {
    work(i) = avail[i];
    for(j=0; j<n; j++) {
        finish(j) = 0;
    }
    int found = 0;
    while(found == 0) {
        int found = 0;
        for(i=0; i<n; i++) {
            if(finish(i) == 0) {
                for(j=0; j<m; j++) {
                    if(hold(i)[j] > work(j))
                        break;
                }
                if(j == m) {
                    for(k=0; k<m; k++) {
                        work[k] += alloc[i][k];
                    }
                    safeSeq[found++] = i;
                    finish(i) = 1;
                    found = 1;
                }
            }
        }
        if(found == 0) {
            printf("System is Not in Safe state\n");
            return 0;
        }
        printf("System is in SAFE state.\n");
        printf("Safe Sequence: ");
    }
}

```

```

for(i=0; i<n; i++) {
    printf("P%d", safeSeq[i]);
    if(i != n-1)
        printf(" -> ");
}
printf("\n");
return 0;
}

```

Output

Enter number of processes: 5
 Enter number of resource type
 Enter Allocation Matrix:

0	1	0
---	---	---

2	0	0
---	---	---

3	0	2
---	---	---

2	1	1
---	---	---

0	0	2
---	---	---

Enter Maximum Matrix:

2	5	3
---	---	---

3	2	2
---	---	---

3	0	2
---	---	---

2	2	2
---	---	---

4	3	3
---	---	---

Enter available Resource:

3	3	2
---	---	---

System is in SAFE state.

Safe Sequence: P1 → P3 → P4 → P0 → P2

Tracing:

Need = Max - Alloc.

P0	7	4	3	P3	0	1	1
----	---	---	---	----	---	---	---

P1	1	2	2	P4	4	3	1
----	---	---	---	----	---	---	---

P2	0	0	0
----	---	---	---

Available


```

for (i = 0; i < n; i++) {
    printf("P%d", safe_seq[i]);
    if (i != n - 1)
        printf(" -> ");
}

printf("\n\n");
return 0;
}

```

Output

Enter number of Processes: 5

Enter number of resource type: 3

Enter Allocation Matrix:

2 0 1 0 0

2 0 0 1 0

3 0 2 0 0

2 1 1 0 1 0

2 0 0 2

Enter Maximum Matrix:

2 5 3

3 2 2 0 0

3 0 2

2 2 2 0 0

4 3 3

Enter available Resource:

3 3 2

System is in SAFE state.

Safe Sequence: P2 → P3 → P4 → P0 → P1

Termination:-

Need = Max - Alloc.

P0 7 4 3

P3 0 1 1

Available = 3 3 2

P1 1 2 2

P4 4 3 1

P2 0 0 0

check P1: Need [3 2 2]

compare 1 2 2 $\leq$ 3 3 2

P1 can execute

Allocated = 200 $\Rightarrow$ [8 3 2] + [200]

$\Rightarrow$ [5 3 2]

check P3: Need [0 1 1]

compare 0 1 1 $\leq$ 5 3 2

Allocated 2 1 1 $\Rightarrow$ [5 3 2] + [2 1 1]

$\Rightarrow$ [7 4 3]

check P4: Need [4 3 1]

compare 4 3 1 $\leq$ 7 4 3

Allocated 0 0 2 $\Rightarrow$ [7 4 3] + [0 0 2]

$\Rightarrow$ [7 4 5]

check P0: Need [7 4 3]

compare 7 4 3 $\leq$ 7 4 5

Allocated: 0 1 0 $\Rightarrow$ [7 4 5] + [0 1 0]

$\Rightarrow$ [7 5 5]

check P2: Need [6 0 0]

compare: 6 0 0 $\leq$ 7 5 5

Allocated: 3 0 2 $\Rightarrow$ [7 5 5] + [3 0 2]

$\Rightarrow$ [10 5 7]

For Safe: $\therefore$ P1 $\rightarrow$ P3 $\rightarrow$ P4 $\rightarrow$ P0 $\rightarrow$ P2

* Deadlock:-

→ #include <stdio.h>

int main() {

int n, m, i, j, k;

printf("Processes: ");

scanf("%d", &n);

printf("Resources: ");

scanf("%d", &m);

int alloc[n][m], req[n][m], avail[m];

int finish[n], work[m];

printf("Enter Allocation Matrix: \n");

for (i = 0; i < n; i++)

for (j = 0; j < m; j++)

scanf("%d", &alloc[i][j]);

printf("Enter Request Matrix: \n");

for (i = 0; i < n; i++)

for (j = 0; j < m; j++)

scanf("%d", &req[i][j]);

printf("Enter Available Resources: \n");

for (i = 0; i < m; i++)

for (j = 0; j < m; j++)

scanf("%d", &avail[i]);

work[i] = avail[i];

}

for (i = 0; i < n; i++)

finish[i] = 0;

for (k = 0; k < n; k++)

for (i = 0; i < n; i++)

if (finish[i] == 0)

for (j = 0; j < m; j++)

if (req[i][j] > work[j])

break;


```

if (j == m) {
    for (j = 0; j < m; j++)
        work[j] = alloc[i][j];
    finish[i] = 1;
}
}
}
}

```

```

printf("Deadlock Processes:");
for (i = 0; i < n; i++)
    if (finish[i] == 0)
        printf("P: %d", i);
printf("\n");
return 0;
}

```

Output

Processes: 5

Resources: 3

Enter Allocated Matrix:

0 1 0

2 0 0

3 0 3

2 1 1

0 0 2

Enter Request Matrix:

0 0 0

2 0 2

0 0 0

1 0 0

0 1 0

Enter Available Resources: 0 0 0

Deadlock Process: P4

Q

Drawing:-

check P0 Request [0 0 0]

compare: [0 0 0] $\leq$ [0 0 0]

P0 can Execute

Allocated: [0 1 0] = [0 0 0] + [0 1 0]
 $\Rightarrow$ [0 1 0]

check P2 Request [0 0 0]

P2 can Execute [0 0 0] $\leq$ [0 1 0]

Allocated: [0 3 0] = [0 1 0] + [0 2 0]
 $\Rightarrow$ [3 1 3]

check P3 Request [1 0 0]

P3 can Execute [1 0 0] $\leq$ [3 1 3]

Allocate: [2 1 1] = [3 1 3] + [2 1 1]
 $\Rightarrow$ [5 2 4]

check P1 Request [2 0 2]

P1 can Execute [2 0 2] $\leq$ [5 2 4]

Allocated: [2 0 0] = [5 2 4] + [2 0 0]
 $\Rightarrow$ [7 2 4]

check P4 Request [0 0 5]

P4 can't Execute [0 0 5] $>$ [7 2 4]

So P4 can't Execute

So deadlock appears at P4.

As it can't be killed.

8/5/26

(a) Wort-bild

(b) Best-fit

(c) first bit

→ #include <stdio.h>

```
void firstfirst (int blockSize, int blocks, int  
processSize, int processes) {
```

and allocation processes:

```
for (int i = 0; i < processes; i++)
```

allocation $(i) = -1$

for $i = 0$ to $n-1$ processes $i++$)

for (int y = 0; y < vops; y++) {

if (blockSize > 0) r = processSize(1)

allocation list:

Wort für 43 = dreissig (1)

week:

```
print("In first let allocation"):
```

```
printf("Process no. %d process size is %d\n",  
       Block.no, n);
```

for $i \in [0, i_{\text{process}} + 1)$

```
print("1. d'lt'1. d'lt'" with processsing)
```

if $\text{AllValid}(i) = \text{true}$

print fn. dir() allocation (i+1);

else

print ("NOT ALL OBJECTS")


```

void bestFit (int blockSize, int blocks, int processes,
             int allocation) {
    for (int i = 0; i < processes; i++) {
        allocation[i] = -1;
        for (int j = 0; j < blocks; j++) {
            if (blockSize[j] == processSize[i]) {
                if (bestIdx == -1 || blockSize[j] < blockSize[bestIdx]) {
                    bestIdx = j;
                }
            }
        }
        if (bestIdx != -1) {
            allocation[i] = bestIdx;
            blockSize[bestIdx] = processSize[i];
        }
    }
    printf("\n Best fit allocation\n");
    printf("Process no. | Process size | block No.\n");
    for (int i = 0; i < processes; i++) {
        printf("%d | %d | %d\n", i+1, processSize[i], allocation[i]);
        if (allocation[i] != -1) {
            printf("%d | %d\n", i+1, allocation[i]);
        }
    }
    printf("\n Not allocated\n");
}

void worstFit (int blockSize, int blocks,
              int processSize, int processes) {

```



```

int Allocation(processes):
for (int i = 0; i < processes; i++)
{
    int workIdn = -1;
    for (int j = 0; j < blocks; j++)
    {
        if (blockSize[j] >= processSize[i])
        {
            if (workIdn == -1 || blockSize[j] > block
                - size(workIdn))
                workIdn = j;
        }
    }
    if (workIdn != -1)
    {
        allocation[i] = workIdn;
        blockSize[workIdn] = processSize[i];
    }
}

printf("\n Work fit Allocation\n");
printf("Process no. | process size | block no.\n");
int main()
{
    int blocks, processes;
    printf("Enter number of memory blocks:\n");
    scanf("%d", &blocks);
    int blockSize[blocks];
    printf("Enter size of memory blocks:\n");
    for (int i = 0; i < blocks; i++)
        scanf("%d", &blockSize[i]);
    printf("Enter no. of processes:\n");
    scanf("%d", &processes);
    int processSize[processes];
    printf("Enter size of processes:\n");

```



```
for (int i = 0; i < blocks; i++)
```

```
{
    blocks[i] = blocksize(i);
```

```
    blocks2[i] = blocksize(i);
```

```
    blocks3[i] = blocksize(i);
```

```
    first fit (block 2, blocks, processsize, process);
```

```
    best fit (block 2, blocks, processsize, process);
```

```
    worst fit (block 3, blocks, processsize, process);
```

```
    return 0;
```

```
}
```

Output

Enter no. of memory blocks = 5

Enter size of memory block:

200 500 200 300 600

Enter number of process: 4

Enter size of 4 processes:

212 417 112 426

--- first fit ---

Process no.	Process size	Block No
1	212	2
2	417	5
3	112	2
4	426	Not allocated

--- Best fit ---

Process No	Process size	Block No
1	212	4
2	417	2
3	112	3
4	426	5

←

-- Wagon bit --

Proven No	Proven Size	Block No
-----------	-------------	----------

1

212

5

2

417

2

3

112

5

7

926

Not Allocated.

Q.1) LRU, optimal and FIFO

→ #include <stdio.h>

void fifo(int pages[], int n, int frames)

{ int frame[20], i, j, k = 0, fault = 0, found;

for (i = 0; i < frames; i++)

frame[i] = -1;

printf("In FIFO Page Replacement\n");

for (i = 0; i < n; i++)

{

found = 0;

for (j = 0; j < frames; j++)

{ if (frame[j] == pages[i])

{

found = 1; break;

break;

}

}

if (!found)

{

frame[k] = pages[i];

k = (k + 1) % frames;

fault++;

}

printf("In d-1) %d pages[i]\n",

for (j = 0; j < frames; j++)

{

if (frame[j] != -1)

printf("In d-1) %d frame[j]\n",

else

printf("-\n");


```

}
}
print("Initial Page Faults = %d\n", fault);

```

```

void printPages(), int n, int frames)
{

```

```

    int frame[10], time[10];

```

```

    int i, j, pos = -1, fault = 0, found;

```

```

    int min, counter = 0;

```

```

    for (i = 0; i < frames; i++)
    {

```

```

        frame[i] = -1;

```

```

        time[i] = 0;
    }
}

```

```

print("Initial Page Replacement\n");

```

```

for (i = 0; i < n; i++)
{

```

```

    found = 0;

```

```

    for (j = 0; j < frames; j++)
    {

```

```

        if (frame[j] == pages[i])
        {

```

```

            counter++;

```

```

            time[j] = counter;

```

```

            found = 1;

```

```

            break;
        }
    }
}

```

```

if (!found)
{

```

```

    pos = -1;
}

```



```
for (j=0; j < frames; j++)
```

```
{
    if (frame[j] != -1)
```

```
    pos = j;
```

```
    break;
```

```
}
```

```
}
```

```
if (pos == -1)
```

```
{
    min = time[0];
```

```
    pos = 0;
```

```
    for (j=1; j < frames; j++)
```

```
    {
        if (time[j] < min)
```

```
        min = time[j];
```

```
        pos = j;
```

```
    }
```

```
}
```

```
counter = 0;
```

```
frame[pos] = pages[i];
```

```
time[pos] = counter;
```

```
faults++;
```

```
}
```

```
printf("In %d \n", pages[i]);
```

```
for (j=0; j < frames; j++)
```

```
{
    if (frame[j] != -1)
```

```
        printf("%d \n", frame[j]);
```

```
333
```



```

counter++;
frame[pos] = pages[i];
time[pos] = counter;
fault++;
}
printf("In %d -> ", pages[i]);
for (j = 0; j < frames; j++)
    if (frame[j] != -1)
        printf(" %d ", frame[j]);
    else
        printf(" - ");
}
}

```

```

printf("Total Page Faults = %d\n", fault);
}

```

void Optional(int pages[], int n, int frames)

```

int frame[20];
int i, j, n, pos, fault = 0, found;
int start, end;

```

```

for (i = 0; i < frames; i++)
    frame[i] = -1;

```

```

printf("In Optional Page Replacement\n");
for (i = 0; i < n; i++)

```

```

    found = 0;

```

```

    for (j = 0; j < frames; j++)

```

```

        if (frame[j] == pages[i])

```

```

            found = 1;

```


break;

}

}

is "found")

pos = -1;

for (j = 0; j < frames; j++)

{ if (frame[j] == -1)

{ pos = j;
break;

}

if (pos == -1)

failed = -1;

for (j = 0; j < frames; j++)

{ if (j == 0 && k == 1)
for (k = i + 1; k < n; k++)

{ if (frame[j] == pages[k])

{ index = k;
break;

}

}

if (index == -1)

{ pos = j;
break;

}


```
if (index > faultless)
{
```

```
    faultless = index;
```

```
    pos = j;
```

```
}
```

```
}
```

```
frame[pos] = pages[i];
fault++;
```

```
}
```

```
printf("\n%d", pages[i]);
```

```
for (j = 0; j < frame; j++)
```

```
    if (frame[j] != -1)
```

```
        printf("%d ", frame[j]);
```

```
    else
```

```
        printf("- ");
```

```
}
```

```
}
```

```
printf("\n Total Page Faults = %d\n", fault);
```

```
end main();
```

```
}
```

```
int pages[50], n, frames, choice;
```

```
printf("Enter number of pages: ");
```

```
scanf("%d", &n);
```

```
printf("Enter Page Reference String: ");
```

```
for (i = 0; i < n; i++)
```

```
    scanf("%d", &pages[i]);
```

```
printf("Enter number of frames: ");
```

```
scanf("%d", &frames);
```

```
printf("\n\n");
```



```

print("h2. LRU");
print("n3. Optional");
print("Enter your choice: ");
scanf("%d", &choice);
printf("choice");
}

```

Call 1:

fifo(pages, n, frames);

break;

Call 2:

lru(pages, n, frames);

break;

Call 3:

optimal(pages, n, frames);

break;

default:

printf("Invalid choice");

return 0;

Output

Enter number of pages: 7

Enter page reference string: 1 2 3 4 5 6 7

Enter number of frames: 3

1. FIFO

2. LRU

3. ~~Optional~~

Enter your choice: 2

LRU page Replacement

1 -> 1 -

2 -> 1 2 -

3 -> 1 2 3 -

4 → 4 2 3

5 → 4 5 3

6 → 4 5 6

7 → 7 5 6

Total Page Faults = 7

--- FIFO ---

1 → 1 -

2 → 1 2 -

3 → 1 2 3

4 → 4 2 3

5 → 4 5 3

6 → 4 5 6

7 → 7 5 6

Total Page Faults = 7

--- Optimal Page Replacement ---

1 → 1 - -

2 → 1 2 -

3 → 1 2 3

4 → 4 2 3

5 → ~~4~~ 5 2 3

6 → ~~4~~ 6 2 3

7 → ~~4~~ 7 2 3

Total Page Faults = 7

15/12/20